CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1 – CO1

Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Total Marks	50
Scenario	Autonomous hospital medicine-delivery robot
Search strategies studied	Breadth-First Search (BFS), Recursive Depth-Limited Search (DLS), Uniform-Cost Search (UCS)

Note: This answer follows the assignment brief. Theoretical results are derived below. The Python appendix provides reproducible tests and benchmark code; measured performance values should be generated on the computer used for submission rather than invented.

TASK 1 – Design the Agent and the Search Algorithms

1.1 PEAS description

PEAS element	Description
Performance measure	Deliver the correct medicine to the requested ward safely, minimise travel time and route cost, avoid blocked corridors, and complete delivery reliably.
Environment	Hospital corridors, junctions, lifts, pharmacy, ward doors, people, other robots and temporary obstacles.
Actuators	Drive motors, steering, lift interface, door interface, display/alert and medicine compartment control.
Sensors	Localisation sensor, wheel encoders, cameras or distance sensors, obstacle sensors, lift/door status and destination input.

1.2 Environment classification

Property	Classification and reason
Observability	Normally partially observable: sensors provide information about location and obstacles, but may be incomplete or fail. With reliable localisation it can be treated as fully observable for the basic model.
Determinism	Mostly deterministic in the simplified graph model: an action moves the robot to a known connected location. Real traffic, people and lift delays introduce uncertainty.
Episodic/sequential	Sequential: each movement affects the robot's next location and future choices.
Static/dynamic	Dynamic: people, carts, doors and corridor conditions can change while the robot plans or moves.
Discrete/continuous	The planning model is discrete because locations and actions are graph nodes and edges. Physical motion and time are continuous in reality.
Agents	Multi-agent environment in practice because people and possibly other robots share the hospital. The simplified route planner can model them as changing obstacles.

1.3 AI view and rationality

The robot follows the **acting rationally** view of AI. It selects actions that are expected to achieve the goal while minimising travel cost and respecting safety constraints. It is a rational problem-solving agent because it uses percepts, a goal and a model of available actions to choose a route that best satisfies its performance measure. Early AI problem solving commonly represented tasks as states and operators and searched a state space for a goal; this hospital route problem follows that classical approach.

1.4 Problem formulation

• **States:** the robot's current graph location, represented by a node identifier.

• **Initial state:** the central pharmacy node.

• **Actions:** move along any edge connecting the current node to a neighbouring junction, corridor, lift or ward door.

• **Transition model:** Result(s, a) is the neighbouring node reached by taking action a from state s.

• **Goal test:** the current node is the requested ward node.

• **Path cost:** sum of travel-time costs on all edges in the route.

Let G = (V,E) be the hospital graph, where V is the set of locations and each edge (u,v) has non-negative travel-time cost c(u,v). The objective is to find a path from the pharmacy s to the requested ward g.

1.5 Three distinct uninformed search strategies

A. Breadth-First Search (iterative, queue)

BFS expands the shallowest unexpanded node first. It uses a FIFO queue and an explored set to avoid repeatedly expanding the same location. It is appropriate as a baseline when each move has equal cost and a route with the fewest edges is desired. Because the assignment gives travel-time costs, BFS is not guaranteed to find the minimum-time route when edge costs differ.

```
BFS(start, goal):
    queue = FIFO queue containing start
    parent[start] = NIL
    explored = empty set
    while queue is not empty:
        u = queue.pop_front()
        if u == goal: return reconstruct(parent, goal)
        if u in explored: continue
        add u to explored
        for each neighbour v of u:
            if v not in explored and v not in parent:
                parent[v] = u
                queue.push_back(v)
    return FAILURE
```

Preprocessing: build an adjacency list from the hospital map; initialise the queue, parent map and explored set. Adjacency-list construction takes O(V+E) time and O(V+E) space.

B. Recursive Depth-Limited Search (recursive, stack)

DLS explores one branch deeply before backtracking, but it stops at a chosen depth limit L. The recursive call stack stores the current path. A path set prevents cycles on the current route. DLS is useful when memory is restricted and a reasonable depth limit is known. It is not complete if the goal is deeper than L.

```
DLS(node, goal, limit, path):
    if node == goal: return SUCCESS
    if limit == 0: return CUTOFF
    cutoff_occurred = false
    for each neighbour v of node:
        if v not in path:
            add v to path
            result = DLS(v, goal, limit - 1, path)
            if result is SUCCESS: return SUCCESS
            remove v from path
            if result is CUTOFF: cutoff_occurred = true
    if cutoff_occurred: return CUTOFF
    return FAILURE
```

Preprocessing: adjacency list and a path set. In a graph with cycles, path checking is essential. A global explored set can be added, but care is needed because reaching a node with a different remaining depth may change the result.

C. Uniform-Cost Search (iterative, priority queue)

UCS expands the frontier node with the smallest accumulated path cost g(n). It uses a min-priority queue and stores the best known cost to each node. With non-negative edge costs, UCS is complete under the usual finite-branching and positive-cost assumptions, and it returns a minimum-cost route. This is the most directly suitable of the three methods for travel-time-weighted corridors.

```
UCS(start, goal):
    priority_queue = [(0, start)]
    best_cost[start] = 0
    parent[start] = NIL
    while priority_queue is not empty:
        (cost, u) = remove_min(priority_queue)
        if cost != best_cost[u]: continue
        if u == goal: return reconstruct(parent, goal), cost
        for each edge (u, v) with cost w:
            new_cost = cost + w
            if v not in best_cost or new_cost < best_cost[v]:
                best_cost[v] = new_cost
                parent[v] = u
                insert_or_decrease_key(v, new_cost)
    return FAILURE
```

Preprocessing: adjacency list, min-priority queue, best-cost map and parent map. Stale priority-queue entries are ignored when removed. All edge costs must be non-negative.

TASK 2 – Identify the Basic Operation

Algorithm	Basic operation	Where and why it dominates
BFS	Node expansion / successor generation	For each dequeued node, BFS examines its outgoing edges and inserts undiscovered neighbours. The total number of examined vertices and edges grows with the explored region.
Recursive DLS	Recursive call and successor examination	Each call checks a node and may invoke the procedure for up to b children. As depth limit or branching factor grows, the number of calls grows as a search tree.
UCS	Priority-queue remove-min and relaxation	Each expansion removes the lowest-cost frontier entry and checks adjacent edges. Heap operations add a logarithmic factor.

These operations are suitable because they are repeated in the main search loop or recursive body and account for most of the algorithm's work. As the state space grows, more nodes and edges may be examined before the goal is removed from the frontier.

TASK 3 – Derive Time and Space Complexity Mathematically

Use b for branching factor, d for the depth of the shallowest goal, m for maximum depth, C* for optimal solution cost and ε for the minimum positive step cost. Let V be the number of graph vertices and E the number of edges. In a tree-shaped search, the number of nodes through depth d is:

N(d) = 1 + b + b² + … + b■ = (b^(d+1) – 1)/(b – 1), for b ≠ 1. Thus N(d) = Θ(b■).

3.1 BFS

In the worst case BFS generates all nodes up to depth d: T(d) = Σ(i=0 to d) b■ = (b^(d+1)–1)/(b–1) = Θ(b■). Its frontier can contain the last generated level, giving O(b■) memory; explored and parent records add O(number of discovered states). On an explicit graph, adjacency-list construction costs O(V+E), and BFS traversal costs O(V+E), so total time and space are O(V+E).

3.2 Recursive DLS

At depth limit L, the recursive search may visit a full b-ary tree through depth L:

T(L) = Σ(i=0 to L) b■ = (b^(L+1)–1)/(b–1) = Θ(b■), for b > 1.

The active recursion path stores at most L+1 nodes, so auxiliary stack/path memory is O(L). If a full explored set is used, memory can rise to O(V). For graph input, include O(V+E) adjacency-list preprocessing; overall time is O(V+E + b■) and memory is O(V+E+L) when the graph and adjacency list are stored.

3.3 UCS

UCS expands nodes in non-decreasing path-cost order. With minimum step cost ε, any path with cost at most C* has at most floor(C*/ε) steps. Define C = floor(C*/ε). A tree-style upper bound on generated nodes is Σ(i=0 to C) b■ = O(b^(C+1)). With a binary heap on an explicit graph, each vertex/edge may be processed with priority-queue operations, giving O((V+E) log V) time and O(V+E) memory for a standard adjacency-list implementation (assuming appropriate decrease-key or stale-entry handling).

Important distinction: the state-space tree bounds describe search-tree growth; the O(V+E) or O((V+E)logV) graph bounds rely on duplicate detection and a finite explicit graph. The hospital map is naturally represented as a graph, so graph-search bounds are the more practical implementation model.

TASK 4 – Best, Average and Worst-Case Behaviour

Algorithm	Best case	Average case	Worst case
BFS	Goal is the start or first shallow node; very few nodes are expanded.	Goal appears among nodes at depth d; a substantial fraction of levels 0…d is explored.	Goal is last at depth d or unreachable; all reachable nodes up to the search limit are explored. Tree bound Θ(b■).

Algorithm	Best case	Average case	Worst case
DLS	Goal lies on the first branch and at small depth.	Goal is somewhere within limit L; work depends strongly on branch order and depth.	Goal is beyond L or unreachable; a full depth-L tree may be visited, Θ(b■).
UCS	Goal is the start or is reached at very low path cost; few low-cost states are expanded.	Many states with path cost below or equal to the goal cost are expanded.	Goal is unreachable or has high cost; all reachable states may be processed. On a finite graph: O((V+E)logV).

For BFS, branch ordering matters within a level. For DLS, a poor first branch can consume most of the depth-limited search before the goal branch is reached. For UCS, the number of expanded states is governed more by accumulated path costs than by depth: inexpensive detours can be expanded before an expensive direct route.

TASK 5 – Asymptotic Analysis

Algorithm	Time growth	Memory growth	Interpretation
BFS (tree model)	O(b■), Ω(1); worst-level search Θ(b■)	O(b■)	Fast for shallow goals, but the frontier can become very large as depth and branching increase.
DLS (tree model)	O(b■), Ω(1); full depth-limited search Θ(b■)	O(L) stack/path	Uses much less search memory, but can miss a goal beyond the limit and may explore an unhelpful branch.
UCS (finite graph)	O((V+E)logV)	O(V+E)	Cost-optimal for non-negative travel times, but stores frontier and best-cost information.

When the hospital grows from 1,000 to 1,000,000+ locations, a factor of 1,000 increase in states can greatly increase search work and stored records. For BFS, frontier memory may become the first practical limit in broad graphs. DLS controls memory but risks incompleteness. UCS is a strong choice for weighted routes, although its priority queue and cost map still require memory proportional to the explored graph. Efficient map storage, duplicate detection and route replanning are therefore important.

TASK 6 – Formulate the Recurrence Relation

For recursive DLS, let T(L) be the maximum number of calls made with remaining depth L in a full b-branching search tree. At L = 0, the algorithm performs a constant amount of work (goal and cutoff checks): T(0)=c. For L>0, it performs constant local work and may call itself once for each of b successors, each with remaining depth L–1:

T(L) = bT(L–1) + c, for L > 0; T(0)=c.

The original problem size is the remaining depth L. Each recursive subproblem has depth L–1. There are at most b recursive calls, and the work outside those calls (checking the node and iterating over the successor list) is represented by c, or more precisely O(b) if successor iteration is counted separately. Using T(L)=bT(L–1)+O(b) gives the same Θ(b■) result for b>1.

TASK 7 – Solve the Recurrence Relation

Use repeated substitution. Starting with T(L)=bT(L–1)+c:

```
T(L) = b[bT(L-2) + c] + c
     = b^2 T(L-2) + bc + c
     = b^3 T(L-3) + b^2 c + bc + c
     ...
     = b^L T(0) + c(1 + b + b^2 + ... + b^(L-1))
     = b^L c + c[(b^L - 1)/(b - 1)]
```

For b>1, the dominant term is b■, so T(L)=Θ(b■). Substitution is applicable because each level reduces the remaining depth by one and produces at most b subproblems of equal maximum size. This shows that recursive DLS can grow exponentially with depth even though its stack memory is only O(L).

Extension to iterative deepening: IDDFS runs DLS repeatedly with limits 0,1,…,d. Its total node count is approximately Σ(i=0 to d)(d–i+1)b■, which remains Θ(b■) for b>1. Upper-level nodes are revisited, but the deepest level dominates the total work. Its memory remains O(d). IDDFS is a useful alternative when the goal depth is unknown and memory is limited; it is not one of the three benchmark algorithms selected above.

TASK 8 – Implement and Test the Algorithms

The following Python code provides compact reference implementations. It uses an adjacency list and non-negative edge weights. BFS returns a minimum-edge route, DLS returns a route within its depth limit, and UCS returns a minimum-travel-time route.

```
from collections import deque
import heapq

def path_from_parent(parent, goal):
    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]
    return list(reversed(path))

def bfs(graph, start, goal):
    q = deque([start])
    parent = {start: None}
    expanded = 0
    while q:
        u = q.popleft()
        expanded += 1
        if u == goal:
            return path_from_parent(parent, goal), expanded
        for v, _ in graph.get(u, []):
            if v not in parent:
                parent[v] = u
                q.append(v)
    return None, expanded

def dls(graph, start, goal, limit):
    expanded = 0
    def visit(u, remaining, path):
        nonlocal expanded
        expanded += 1
        if u == goal:
            return path[:]
        if remaining == 0:
            return None
        for v, _ in graph.get(u, []):
            if v not in path:
                path.append(v)
                result = visit(v, remaining - 1, path)
                if result is not None:
                    return result
                path.pop()
        return None
    return visit(start, limit, [start]), expanded

def ucs(graph, start, goal):
    pq = [(0, start)]
    best = {start: 0}
    parent = {start: None}
    expanded = 0
    while pq:
        cost, u = heapq.heappop(pq)
        if cost != best.get(u):
            continue
        expanded += 1
        if u == goal:
            return path_from_parent(parent, goal), cost, expanded
        for v, weight in graph.get(u, []):
            new_cost = cost + weight
            if new_cost < best.get(v, float("inf")):
                best[v] = new_cost
                parent[v] = u
                heapq.heappush(pq, (new_cost, v))
    return None, float("inf"), expanded

def make_test_graph(n):
    # A connected chain with an extra two-step shortcut edge pattern.
    # Edge weights are positive; deterministic and reproducible.
    g = {i: [] for i in range(n)}
    for i in range(n - 1):
```

```
            g[i].append((i + 1, 2))
            g[i + 1].append((i, 2))
        for i in range(n - 3):
            if i % 4 == 0:
                g[i].append((i + 3, 9))
                g[i + 3].append((i, 9))
        return g

if __name__ == "__main__":
    graph = make_test_graph(1000)
    print("Reachable:", bfs(graph, 0, 999)[0] is not None)
    print("Start equals goal:", bfs(graph, 5, 5)[0])
    # Test unreachable goal by omitting an isolated goal node.
    graph[1000] = []
    print("Unreachable:", bfs(graph, 0, 1000)[0])
    print("DLS within limit:", dls(graph, 0, 10, 10)[0] is not None)
    print("UCS cost:", ucs(graph, 0, 999)[1])
```

Test cases and expected behaviour:

• Reachable goal: start at node 0 and goal at node 999; all three algorithms should return a route (DLS needs a sufficient depth limit).

• Unreachable goal: add an isolated node 1000; each algorithm should return failure/None.

• Start equals goal: start and goal are both node 5; each algorithm should return the one-node route [5] with zero route cost for UCS.

For the assignment's scale tests, call make_test_graph(n) for n = 1,000, 10,000, 100,000 and 1,000,000. Use identical start/goal pairs and at least five runs per algorithm. Report the Python version, machine specifications, random seed if random graph generation is added, and the depth limit used for DLS. A chain graph is deliberately simple and does not represent every hospital topology; add a second reproducible branching graph if time permits. Python's recursion limit may prevent recursive DLS from traversing extremely deep paths; record this as an implementation limitation rather than silently changing the algorithm.

TASK 9 – Record Experimental Performance

Record the following for each graph size and each of the five or more runs: nodes generated, nodes expanded, maximum frontier size, solution depth, path cost, optimality and elapsed time. Use the mean execution time across runs. The table below is a results template; fill it with the actual output from the provided implementation on your computer.

States	Algorithm	Generated	Expanded	Max frontier	Depth / cost	Mean time (s)
1,000	BFS / DLS / UCS	Measure	Measure	Measure	Measure	5-run mean
10,000	BFS / DLS / UCS	Measure	Measure	Measure	Measure	5-run mean
100,000	BFS / DLS / UCS	Measure	Measure	Measure	Measure	5-run mean
1,000,000	BFS / DLS / UCS	Measure	Measure	Measure	Measure	5-run mean

To measure frontier size and generated nodes precisely, add counters at the point where a previously unseen state is inserted into the frontier, and update max_frontier after each insertion. In UCS, count successful relaxations separately from heap pushes because stale entries may later be discarded. Use time.perf_counter() around only the search call, not graph construction, and state clearly whether preprocessing time is reported separately or included.

TASK 10 – Compare Theory with Experimental Behaviour

• If BFS node counts rise with the number of reachable states, this is consistent with its O(V+E) graph-search bound. Its frontier size depends on graph shape and can be large in wide layers.

• DLS may expand fewer nodes when the goal is found early on the first explored branch, but can revisit many nodes or fail when the limit is too small. Its recursion stack grows with depth, not with the total number of states.

• UCS may expand more nodes than BFS if many low-cost detours exist, but it respects edge costs and returns an optimal travel-time route for non-negative costs.

• Measured times may differ from asymptotic predictions because of Python interpreter overhead, heap operations, memory allocation, graph degree, duplicate edges, tie-breaking and operating-system load.

• Do not claim one algorithm scaled best from a single graph. The conclusion must be based on repeated measurements over the same graph instances and start/goal pairs.

TASK 11 – Recommend and Defend the Final Design

Recommend **Uniform-Cost Search** for the original hospital route-planning requirement because each movement has a travel-time cost and the objective is to minimise total travel time. BFS minimises the number of edges only when all moves have equal cost. DLS uses little memory but depends on a depth limit, may not find the goal and does not guarantee minimum cost.

UCS is complete and optimal for a finite graph with non-negative edge costs. Its standard graph implementation uses O((V+E)logV) time and O(V+E) storage. The trade-off is that it maintains a priority queue and best-cost records. For a hospital with 1,000,000+ states, use a compact adjacency representation, efficient priority queue, duplicate detection, map updates and local replanning when corridors change. The final recommendation should be confirmed by the measured results from Task 9; if the actual graph and resource limits make UCS impractical, report the measured limitation and justify a revised design.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design

A. Impact analysis

The original problem assumed a known start location. After a sensor fault, the robot only knows that it is at one of k possible junctions. The environment is now partially observable and the exact physical state is uncertain. The robot's knowledge becomes part of the planning state because the planner must represent all locations that are still possible, rather than pretending that one location is certain.

B. Evaluation of the original design

The original BFS, DLS or UCS algorithms can still be used if the planner's state is changed from one location to a **belief state** (a set of possible locations). A belief state B is a subset of V. A conservative upper bound on the number of possible belief states is 2^V; if the robot knows there are exactly k candidate locations at one time, the number of subsets of those candidates is 2^k. This can grow quickly: for k=2 there are 4 subsets, k=4 gives 16, and k=8 gives 256. Only non-empty beliefs may be relevant, so subtract one for the empty set when appropriate.

C. Adaptation / redesign

Modify the design to use belief-state search with local sensing. The initial belief is B■={v■,…,v■}. A movement action updates every possible location according to the transition model, and the resulting belief is the union of all possible successor locations. A local sensor observation filters the belief to locations consistent with that observation. Plan toward the pharmacy docking station while ensuring that the action is safe for all locations still considered possible. This is more robust than planning from an assumed single node.

The cost of searching over beliefs can be exponential in the number of candidate states because the belief space may contain up to 2^k subsets. Local sensing can reduce the belief size and therefore reduce practical search. A conditional plan can choose different later actions depending on the sensor observation.

D. Evidence-based defence and small experiment

Candidate locations k	All subsets 2^k	Non-empty subsets 2^k–1	With perfect local identification
2	4	3	Belief becomes a singleton: 1 possibility
4	16	15	Belief becomes a singleton: 1 possibility
8	256	255	Belief becomes a singleton: 1 possibility

For a small reproducible experiment, enumerate the non-empty subsets of k candidate locations and compare the number of possible beliefs before and after a perfect local identification sensor. Without a sensor, the robot may need

to retain a multi-location belief. With a perfect sensor, the belief is reduced to one actual location, after which the original UCS route planner can be applied from that location. The following short code verifies the subset counts:

```
from itertools import combinations

def nonempty_beliefs(k):
    count = 0
    for r in range(1, k + 1):
        count += sum(1 for _ in combinations(range(k), r))
    return count

for k in (2, 4, 8):
    print(k, 2**k, nonempty_beliefs(k), "after perfect sensor:", 1)
```

Expected output:

```
2 4 3 after perfect sensor: 1
4 16 15 after perfect sensor: 1
8 256 255 after perfect sensor: 1
```

This is a count of possible belief sets, not a measurement of travel distance. To measure plan length, build a small sample hospital graph, define a goal node (pharmacy), and calculate the worst-case conditional plan over all candidate starting nodes. Report the exact graph, sensor model and plan-length definition. A perfect sensor is an explicit simplifying assumption; a real sensor may be noisy and may leave more than one possible location.

Overall Conclusion

For the original fully localised, weighted hospital map, UCS is the best of the three selected strategies because it finds a minimum-travel-time route when edge costs are non-negative. BFS is a useful unweighted baseline, while recursive DLS demonstrates low-memory depth-limited search but has completeness and optimality limitations. When localisation becomes uncertain, the design should be modified to plan over belief states and use local sensing to reduce uncertainty. The final implementation should be validated with repeatable experiments at all four required graph sizes.

Submission checklist

• Add your name, register number, department and submission date to the cover page if required.

• Run the code on the four requested state-space sizes and replace all 'Measure' cells with actual results.

• Capture terminal outputs and create graphs for time, expanded nodes and maximum frontier size; use a logarithmic axis if values span a wide range.

• State the machine configuration, Python version, graph-generation method, random seed and DLS depth limit.

• Check that the experimental graph is identical across algorithms and that the reported route cost and optimality are correct.